

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

КАТЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Алгоритми та методи обчислень

Лабораторна робота №3

«Інтерполяція функцій»

Виконав:
студент групи ІО-32
Крадожон М.
Перевірів(-ла):
Порєв В.

Київ – 2023

Лабораторна робота №2

Тема: «Інтерполяція функцій»

Мета: Ознайомлення з інтерполяційними формулами Лагранжа, Ньютона, рекурентним співвідношенням Ейткена, методами оцінки похибки інтерполяції.

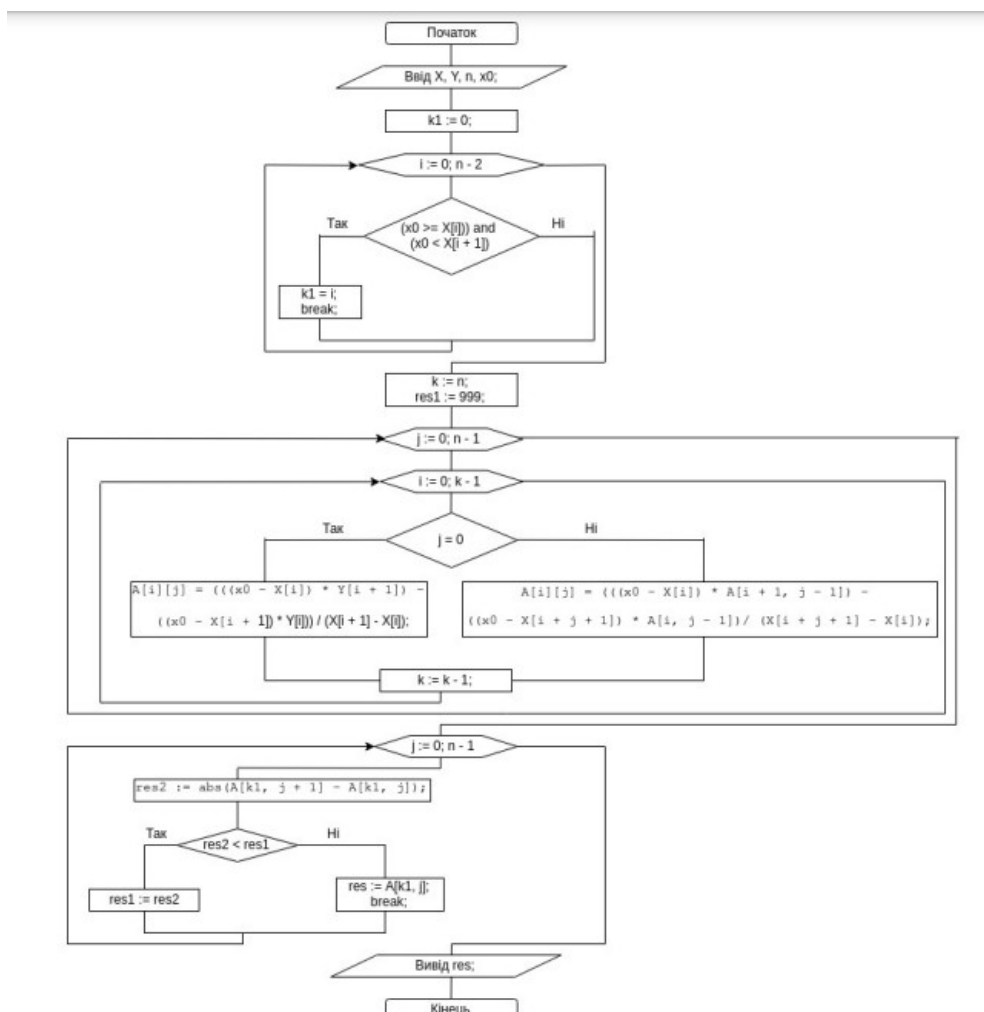
Завдання: Закріплення, поглиблення і розширення знань студентів при вирішенні практичних обчислювальних завдань. Оволодіння обчислювальними методами і практичними методами оцінки похибки обчислень. Придбання умінь і навичок при програмуванні та налагодженні обчислювальних завдань на комп'ютері.

Варіант: 11

<u>11</u>	$1/(1 + e^{-x})$	[0, 4]	1.8	12	$\cos(2x + x^2)$	[0, 1]	1.8
-----------	------------------	--------	-----	----	------------------	--------	-----

$$f(x) \approx M_{0,1,\dots,i}(x) = \frac{1}{x_i - x_0} \begin{vmatrix} x - x_0 & M_{0,1,\dots,i-1}(x) \\ x_i - x_0 & M_{0,1,\dots,i}(x) \end{vmatrix} \quad (1.8)$$

Блок-схеми алгоритмів:



Код програми:

Алгоритм

```
const calculateAiken = (a: number, b: number, n: number) => {
  if (
    isNaN(a) ||
    isNaN(b) ||
    isNaN(n) ||
    a >= b ||
    n < 2 ||
    n > 1000 ||
    a < -100 ||
    b > 100 ||
    a > 100 ||
    b < -100
  ) {
    alert('Введіть коректні дані')
    return null
  }

  const h = (b - a) / (n - 1)
  const xValue = Array.from({ length: n }, (_, i) => a + i * h)
  const y = xValue.map((x) => 1 / (1 + Math.exp(-x)))

  const getInterpolatedValue = (x: number) => {
    const buffer = [...y]
    for (let i = 0; i < n - 1; i++) {
      for (let j = i + 1; j < n; j++) {
        buffer[j] =
          ((x - xValue[i]) * buffer[j] - (x - xValue[j]) * buffer[i]) /
          (xValue[j] - xValue[i])
      }
    }
    return buffer[n - 1]
  }

  const yPoli = xValue.map((x) => getInterpolatedValue(x))
  const delta = xValue.map((x) =>
    Math.abs(Math.exp(Math.cos(x)) - getInterpolatedValue(x))
  )

  return { xValue, y, yPoli, delta }
}

export { calculateAiken }
```

Програма та графічний інтерфейс

App.tsx

```
import Header from './components/Header'
import FormComp from './components/FormComp'
import Charts from './components/Charts'

export default function App() {
  return (
    <main className="flex flex-col gap-5 p-5">
      <Header />
      <FormComp />
      <Charts />
    </main>
  )
}
```

Header.tsx

```
export default function Header() {
  return (
    <>
      <h1 className="w-full text-center text-3xl font-extrabold">
        Метод Ейткена
      </h1>
      
    </>
  )
}
```

FormComp.tsx

```
import { useEffect } from 'react'
import { useForm } from 'react-hook-form'
import { useSearchParams } from 'react-router-dom'

import { calculateAiken } from '@/lib/aiken'
import { useChartContext } from '@/common/context/ChartContext'

import { Label } from '../ui/label'
import { Input } from '../ui/input'
import { Button } from '../ui/button'

export default function FormComp() {
  const { chartsVisible, setChartsVisible, setChartData } = useChartContext()
  const [searchParams, setSearchParams] = useSearchParams()

  const { register, handleSubmit, setValue } = useForm({
    shouldUseNativeValidation: true,
    defaultValues: {
      a: searchParams.get('a') ? Number(searchParams.get('a')) : 0,
      b: searchParams.get('b') ? Number(searchParams.get('b')) : 4,
      n: searchParams.get('n') ? Number(searchParams.get('n')) : 10
    }
  })
}
```

```

    }
  })

  useEffect(() => {
    const a = searchParams.get('a')
    const b = searchParams.get('b')
    const n = searchParams.get('n')

    if (a) setValue('a', Number(a))
    if (b) setValue('b', Number(b))
    if (n) setValue('n', Number(n))
  }, [searchParams, setValue])

  const onSubmit = async (data: { a: number; b: number; n: number }) => {
    setSearchParams({
      a: String(data.a),
      b: String(data.b),
      n: String(data.n)
    })

    const result = calculateAiken(data.a, data.b, data.n)
    if (result) {
      setChartData(result)
      setChartsVisible(true)
    }
  }

  const handleClear = () => {
    setChartsVisible(false)
    setChartData(null)
  }

  const formFields = [
    { name: 'a', placeholder: 'Значення A' },
    { name: 'b', placeholder: 'Значення B' },
    { name: 'n', placeholder: 'Значення N' }
  ]

  return (
    <form
      onSubmit={handleSubmit(onSubmit)}
      className="flex w-full flex-col items-center gap-5"
    >
      <div className="flex gap-5">
        {formFields.map(({ name, placeholder }) => (
          <div className="flex flex-col gap-1 font-semibold" key={name}>
            <Label htmlFor={name}>{placeholder}</Label>
            <Input
              {...register(name as 'a' | 'b' | 'n', { valueAsNumber: true })}
              type="number"
              placeholder="Число"
              key={name}
              id={`input${name.toUpperCase()}`}>

```

```

    />
  </div>
  )})
</div>

<div className="flex gap-4">
  <Button type="submit">Обчислити</Button>
  {chartsVisible && (
    <Button type="button" variant="secondary" onClick={handleClear}>
      ОЧИСТИТИ
    </Button>
  )}
</div>
</form>
)
}

```

Charts.tsx

```

import { useEffect, useRef, useMemo } from 'react'
import Chart from 'chart.js/auto'

import { getRandomColor } from '@/utils/getRandomColor'
import { useChartContext } from '@/common/context/ChartContext'

export default function Charts() {
  const { chartsVisible, chartData } = useChartContext()

  const chartRef1 = useRef<HTMLCanvasElement>(null)
  const chartRef2 = useRef<HTMLCanvasElement>(null)
  const chartRef3 = useRef<HTMLCanvasElement>(null)
  const chartInstances = useRef<Chart[]>([])

  const chartConfigs = useMemo(
    () => [
      { ref: chartRef1, label: '1/(1+e^-x)', dataKey: 'y' },
      { ref: chartRef2, label: 'Інтерполяція', dataKey: 'yPoli' },
      { ref: chartRef3, label: 'Похибка', dataKey: 'delta' }
    ] as const,
    []
  )

  const processedLabels = useMemo(() => {
    if (!chartData) return null
    return chartData.xValue.map((x: number) => Number(x.toFixed(2)))
  }, [chartData])

  useEffect(() => {
    return () => {
      for (const chart of chartInstances.current) chart.destroy()
      chartInstances.current = []
    }
  })
}

```

```

}, [])

useEffect(() => {
  if (!chartData || !chartsVisible || !processedLabels) return

  for (const chart of chartInstances.current) chart.destroy()
  chartInstances.current = []

  const allRefsAvailable = chartConfigs.every(
    (config) => config.ref.current && config.ref.current.getContext('2d')
  )

  if (!allRefsAvailable) return

  const { xValue } = chartData
  const labels = xValue.map((x: number) => Number(x.toFixed(2)))

  for (const { ref, label, dataKey } of chartConfigs) {
    const ctx = ref.current!.getContext('2d')!

    const chart = new Chart(ctx, {
      type: 'line',
      data: {
        labels,
        datasets: [
          {
            label,
            data: chartData[dataKey],
            borderColor: getRandomColor(),
            borderWidth: 2,
            tension: 0.4
          }
        ]
      },
      options: {
        responsive: true,
        maintainAspectRatio: false,
        scales: {
          y: {
            beginAtZero: true
          }
        }
      }
    })

    chartInstances.current.push(chart)
  }
}, [chartData, chartsVisible, chartConfigs, processedLabels])

if (!chartsVisible || !chartData) return null

return (
  <>

```

```

{chartsVisible && chartData && (
  <>
  <div className="flex h-[50vh] gap-10 px-5">
    {chartConfigs.map(({ ref }, index) => (
      <div className="w-full max-w-1/3" key={index}>
        <canvas ref={ref} id={`diagram${index > 0 ? index + 1 : ''}`} />
      </div>
    ))}
  </div>

  <div className="mt-8 overflow-x-auto px-5">
    <h3 className="mb-3 text-xl font-semibold">Таблиця результатів</h3>
    <table className="min-w-full border border-gray-300">
      <thead className="bg-gray-100">
        <tr>
          {[ 'x', 'Exp(cos(x))', 'Інтерполяція', 'Похибка' ].map(
            (title) => (
              <th className="border border-gray-300 p-2">{title}</th>
            )
          )}
        </tr>
      </thead>
      <tbody>
        {chartData.xValue.map((x: number, index: number) => (
          <tr>
            <td className="border border-gray-300 p-2">
              {x.toFixed(4)}
            </td>
            <td className="border border-gray-300 p-2">
              {chartData.y[index].toFixed(8)}
            </td>
            <td className="border border-gray-300 p-2">
              {chartData.yPoli[index].toFixed(8)}
            </td>
            <td className="border border-gray-300 p-2">
              {chartData.delta[index].toFixed(8)}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</>
)}
</>
)
}

```


Context

ChartContext.tsx

```
import {
  createContext,
  ReactNode,
  SetStateAction,
  useContext,
  useState
} from 'react'

import { ChartData } from '@types/chart-data'

interface ChartContextProps {
  chartsVisible: boolean
  setChartsVisible: React.Dispatch<SetStateAction<boolean>>
  chartData: ChartData | null
  setChartData: React.Dispatch<SetStateAction<ChartData | null>>
}

const ChartContext = createContext<ChartContextProps | undefined>(undefined)

export default function ChartProvider({ children }: { children: ReactNode }) {
  const [chartsVisible, setChartsVisible] = useState(false)
  const [chartData, setChartData] = useState<ChartData | null>(null)

  return (
    <ChartContext.Provider
      value={{
        chartsVisible,
        setChartsVisible,
        chartData,
        setChartData
      }}
    >
      {children}
    </ChartContext.Provider>
  )
}

function useChartContext() {
  const context = useContext(ChartContext)
  if (context === undefined) {
    throw new Error('useChartContext must be used within a ChartProvider')
  }
  return context
}

export { ChartContext, useChartContext }
```

Utils

getRandomColor.tsx

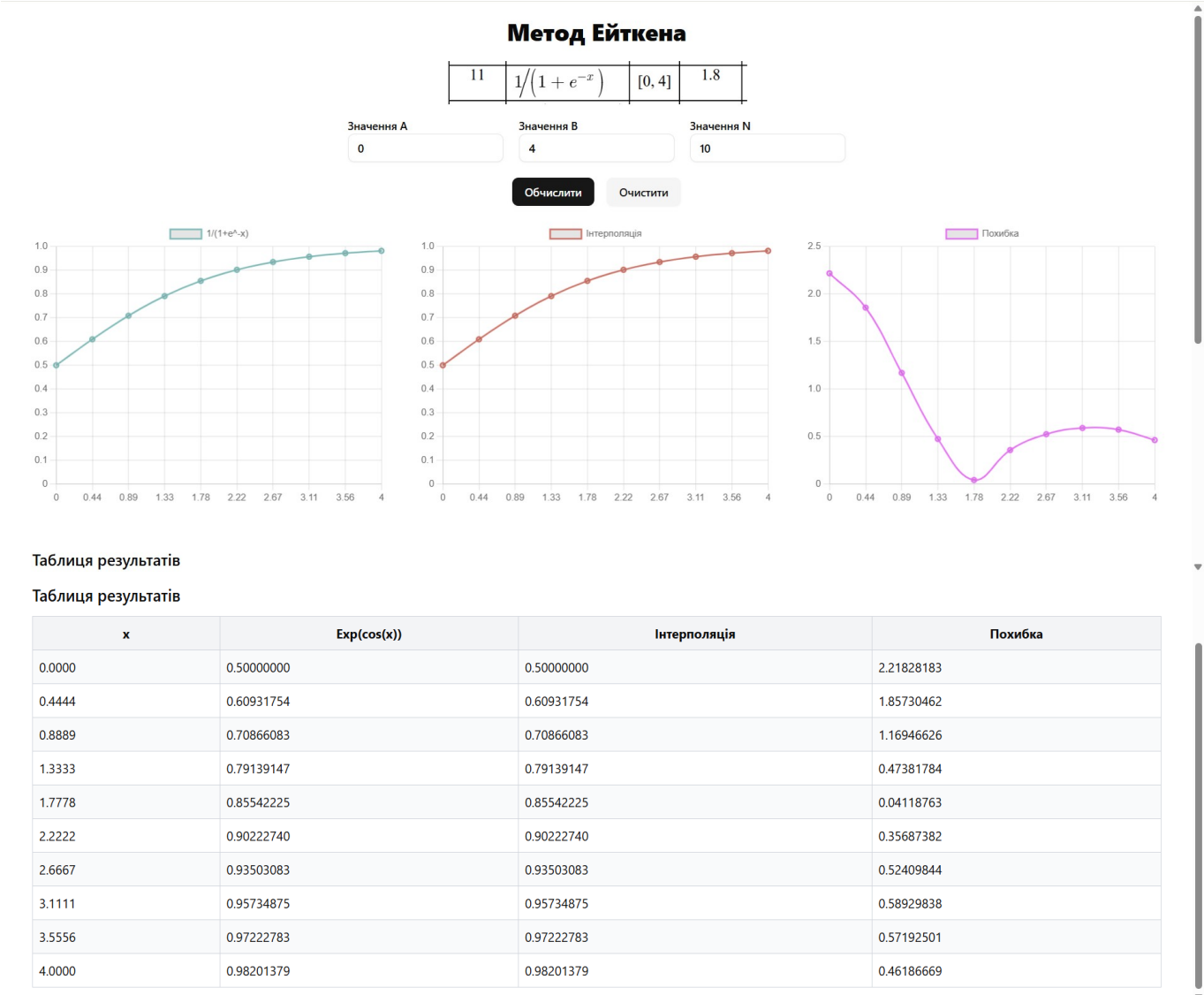
```
const getRandomColor = () => {  
  return `rgba(${Math.floor(Math.random() * 256)}, ${Math.floor(  
    Math.random() * 256  
  )}, ${Math.floor(Math.random() * 256)}, 1)`  
}  
  
export { getRandomColor }
```

Types

chart-data.ts

```
type ChartData = {  
  xValue: number[]  
  y: number[]  
  yPoli: number[]  
  delta: number[]  
}  
  
export { type ChartData }
```

Скриншоти, що підтверджують виконання роботи:



Аналіз результатів:

Завдання полягало в створенні інтерфейсу для інтерполяції функцій та оцінки похибки обчислень. Для реалізації цього завдання використовувалися бібліотеки React та Chart.js для відображення графіків. Інтерфейс дозволяє користувачеві вводити область визначення, кількість значень та побудувати графіки функцій та оцінювати похибку. Після натискання кнопки "Обчислити", відбувається розрахунок та відображення графіків функції, інтерпольованої функції та похибки на окремих графіках.

Введені користувачем дані передаються до відповідних функцій для подальших обчислень та відображення результатів. Загальна структура програми забезпечує зручне та ефективне виконання інтерполяції функцій та оцінки похибки обчислень.

Висновок: У цій роботі було успішно створено інтерфейс для інтерполяції функцій та оцінки похибки обчислень. Користувач може вибрати функцію, ввести область визначення та кількість значень, а програма автоматично розраховує та будує графіки функцій, інтерпольовану функцію та оцінює похибку. Результати відображаються на окремих графіках та в таблиці, забезпечуючи зручний та інформативний інтерфейс для роботи з інтерполяцією функцій. Ця робота демонструє практичне застосування обчислювальних методів у візуалізованій формі, роблячи її корисною для подальшого вивчення та застосування в різних галузях.